

Non-Preemptive Priority Scheduling algorithm

Completed

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8
9     int at[n], bt[n], pr[n];
10    int ct[n], tat[n], wt[n];
11    int completed[n];
12
13    for(int i = 0; i < n; i++) {
14        completed[i] = 0;
15    }
16
17    for(int i = 0; i < n; i++) {
18        printf("Enter AT, BT and Priority for P%d: \n", i+1);
19        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
20    }
21}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

[Back To Practice](#)

< Q1 >

[Save](#)

Non-Preemptive Priority Scheduling algorithm

Completed

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language : C (GCC 9.2.0)

```
22 int time = 0, count = 0;
23 float total_tat = 0, total_wt = 0;
24
25 while(count < n) {
26     int highest_priority = 1000000;
27     int index = -1;
28
29     // Find highest priority process among arrived ones
30     for(int i = 0; i < n; i++) {
31         if(at[i] <= time && completed[i] == 0) {
32             if(pr[i] < highest_priority) {
33                 highest_priority = pr[i];
34                 index = i;
35             }
36             // If priority is same, choose the one with smaller arrival time
37             else if(pr[i] == highest_priority) {
38                 if(at[i] < at[index]) {
39                     index = i;
40                 }
41             }
42         }
43     }
44 }
```

Test Cases 2

[Run Sample Cases](#)[Run All Cases](#)

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

[Back To Practice](#)

< Q1 >

[Save](#)

Non-Preemptive Priority Scheduling algorithm

Completed

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

```
50     tat[index] = ct[index] - at[index];
51     wt[index] = tat[index] - bt[index];
52
53     total_tat += tat[index];
54     total_wt += wt[index];
55
56     completed[index] = 1;
57     count++;
58 }
59 }
60
61 printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
62 printf("Average Waiting Time = %.2f\n", total_wt / n);
63
64 return 0;
65 }
```

[Test Cases](#) 2[Run Sample Cases](#)[Run All Cases](#)

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2